

Trabajo Práctico N° 1: Unidad Aritmético Lógica (ALU)

Arquitectura de Computadoras

Integrantes:

- De la Mata, Nicolás
- Quispe, Mateo

Github: <https://github.com/Tuteku/TP-Arquitectura-comp>

1. Objetivo

El trabajo consistió en implementar una ALU sobre FPGA, parametrizable en el ancho del bus de datos para poder reutilizarla en el resto de trabajos de la materia, y validarla mediante un test bench con generación de entradas aleatorias y chequeo automático de resultados. Adicionalmente se pedía simular el diseño con las herramientas de Vivado, incluyendo el análisis de tiempo.

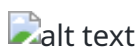
La placa utilizada fue una **Digilent Basys3**, con FPGA Xilinx Artix-7 (`xc7a35tcpg236-1`), y el entorno de desarrollo fue **Vivado 2025.2**.

2. Descripción general del diseño

El sistema se dividió en dos módulos:

- **alu**: la unidad aritmético-lógica propiamente dicha. Es un bloque combinacional que recibe dos operandos y un código de operación, y devuelve el resultado.
- **alu_top**: el nivel superior, que conecta la ALU con la placa física. Contiene tres registros que capturan los operandos y el opcode desde los switches, e instancia la ALU.

2.1 Diagrama de bloques



3. El módulo ALU

3.1 Interfaz

```
module alu #(
    parameter NB_DATA = 8,
    parameter NB_OP    = 6
)
(
    input  wire signed [NB_DATA-1:0] i_a,
    input  wire signed [NB_DATA-1:0] i_b,
    input  wire          [NB_OP-1:0]  i_op,
    output reg signed [NB_DATA-1:0] o_alu
);
```

El parámetro `NB_DATA` define el ancho del bus de datos. `NB_OP` se mantuvo en 6 debido a los códigos de operación provistos por la cátedra.

3.2 Operaciones implementadas

Operación	Código	Descripción
ADD	100000	Suma
SUB	100010	Resta
AND	100100	AND bit a bit
OR	100101	OR bit a bit
XOR	100110	XOR bit a bit
SRA	000011	Desplazamiento derecha aritmético
SRL	000010	Desplazamiento derecha lógico
NOR	100111	NOR bit a bit

4. Decisiones de diseño

4.1 Puertos declarados como `signed`

Los operandos y la salida se declararon `signed`. El desplazamiento aritmético solo extiende el signo si el operando está declarado con `signed`. Sin esa declaración las operaciones SRA y SRL producirían resultados idénticos.

4.2 Cantidad de desplazamiento acotada

En las operaciones de shift se utilizan únicamente los bits menos significativos de `i_b`:

```
localparam NB_SHIFT = $clog2(NB_DATA);
...
SRA : o_alu = i_a >>> i_b[NB_SHIFT-1:0];
SRL : o_alu = i_a >> i_b[NB_SHIFT-1:0];
```

La justificación es que desplazar un dato de 8 bits en 8 o más posiciones produce siempre el mismo resultado, por eso se definió que el campo de desplazamiento son los NB_SHIFT bits bajos de i_b. El ahorro de logica que produce esta decisión se cuantifica en la sección 7.1.

4.3 Comportamiento ante un código de operación inválido

El `case` cubre 8 de los 64 códigos posibles. En el caso del `default` se optó por:

```
default: o_alu = {NB_DATA{1'bx}};
```

El valor `x` cumple dos funciones. En simulación se propaga y hace visible de inmediato cualquier situación en la que se esté decodificando un opcode no previsto. En síntesis, `x` significa “no importa” y le da libertad al optimizador para elegir la implementación que menos lógica consuma.

4.4 Registros con enables independientes

Los tres registros del `alu_top` se cargan mediante tres sentencias `if` independientes.

```
if (i_load_a) r_a <= i_switches;
if (i_load_b) r_b <= i_switches;
if (i_load_c) r_op <= i_switches[NB_OP-1:0];
```

Esto es debido a que en los `if`, cada señal de carga llega directamente con el positivo del clock. Si se activan dos a la vez, ambos registros se cargan con el mismo valor.

4.5 Reset síncrono

Se incorporó un reset síncrono activo en alto, con prioridad sobre las cargas:

```
always @(posedge i_clk) begin
  if (i_reset) begin
    r_a <= {NB_DATA{1'b0}};
    r_b <= {NB_DATA{1'b0}};
    r_op <= {NB_OP{1'b0}};
  end
  else begin
    // cargas
  end
end
```

5. Interfaz con la placa

El módulo `alu_top` conecta el diseño con los periféricos físicos de la Basys3.

Puerto	Pines	Elemento físico
i_switches[7:0]	V2, T3, T2, R3, W2, U1, T1, R2	SW8 a SW15
o_leds[7:0]	V13, V3, W3, U3, P3, N3, P1, L1	LD8 a LD15
i_load_a	U17	BTND
i_load_b	T17	BTNR
i_load_c	W19	BTNL
i_reset	T18	BTNU
i_clk	W5	Oscilador de 100 MHz

6. Verificación

Se desarrollaron dos test benches con propósitos distintos:

Test bench	Módulo bajo prueba	Qué verifica
tb_alu	alu	Las ocho operaciones, con chequeo automático
tb_alu_top	alu_top	La carga de los registros y la integración

6.1 Test bench de la ALU

El test bench de la ALU se organizó en torno a una `task` que encapsula todo el procedimiento de verificación:

```
task alu_task;
  input signed [NB_DATA-1:0] a;
  input signed [NB_DATA-1:0] b;
  input          [NB_OP-1:0] op;
  begin
    i_a = a; i_b = b; i_op = op;
    #1
    case (op)
      ADD : esperado = a + b;
      SUB : esperado = a - b;
      // ...
    endcase
    casos = casos + 1;
    if (o_alu != esperado) begin
      errores = errores + 1;
      $display("ERROR caso %0d: ...", casos, ...);
    end
  end
endtask
```

La `task` recibe los dos operandos y el opcode, los aplica a la ALU, calcula en paralelo el resultado esperado con los operadores propios de Verilog y compara ambos valores con `!=`, que a diferencia de `!=` también distingue los `x`. De esa forma el chequeo es automático y no depende de mirar la forma de onda. Cada llamada incrementa el contador de casos y, si hay un valor distinto, imprime el detalle del caso y suma un error.

Las pruebas se dividen en dos grupos. Primero cuatro casos dirigidos que apuntan a situaciones límite (desbordamiento en la suma, resta que da cero, desplazamiento por cero y un opcode no contemplado), y después un bucle de 100 iteraciones que invoca las ocho operaciones con operandos generados por `$random`, lo que da un total de 804 casos. Al finalizar se imprime el resumen con la cantidad de casos, la cantidad de errores y el veredicto `TEST PASSED / TEST FAILED`.

6.2 Test bench del `alu_top`

El `alu_top` no agrega lógica aritmética, así que este segundo test bench no vuelve a verificar las operaciones, sino la parte secuencial del diseño, que cada registro se cargue únicamente cuando su propia señal de enable está activa, que la carga ocurra con el flanco de clock y que la salida refleje el contenido de los registros. Por eso el chequeo se observa con `$display` en lugar de comparar contra un modelo de referencia.

El banco genera un clock de 10 ns (100 MHz, el mismo del oscilador de la placa) con un `always` y aplica los estímulos dentro de un único bloque `initial`:

```
initial
begin
    #0
    i_clk = 1'b1;
    i_reset = 1'b0;
    i_switches = {NB_DATA{1'b0}};
    i_load_a = 1'b0;
    i_load_b = 1'b0;
    i_load_c = 1'b0;
    @(negedge i_clk); // Cargar registro A
    i_switches = 8'd10;
    i_load_a = 1'b1;
    @(posedge i_clk);
    #1;
    $display("r_a = %d", u_01.r_a);
    @(negedge i_clk); // Cargar registro B
    i_load_a = 0;
    i_load_b = 1;
    i_switches = 8'd5;
    @(posedge i_clk);
    #1;
    $display("r_b = %d", u_01.r_b);
    @(negedge i_clk); // Cargar registro OP
    i_load_b = 0;
    i_switches = 6'b100000;
    i_load_c = 1;
    @(posedge i_clk);
    #1;
    $display("r_op = %b", u_01.r_op);
    #1;
    $display("o_leds = %d", u_01.o_leds);

    @(negedge i_clk);
    i_load_c = 1'b0;
    i_switches = 8'd99;
    @(posedge i_clk);
    #1;
    $display("Con todos los load en 0 y switches=99:");
    $display(" r_a=%0d r_b=%0d r_op=%b", u_01.r_a, u_01.r_b, u_01.r_op);

    #(PERIODO*10) $finish;
end
```

Las entradas se modifican siempre en el flanco negativo (`@(negedge i_clk)`), de manera que llegan estables al flanco positivo y se respetan los tiempos de setup y hold del registro. Las lecturas

se hacen después del flanco positivo con un retardo #1 , porque las asignaciones no bloqueantes del always recién actualizan los registros al final del paso de simulación, si se leyera exactamente en el posedge se vería todavía el valor viejo.

Para observar el estado interno se usa acceso jerárquico a las señales del módulo instanciado (u_01.r_a , u_01.r_b , u_01.r_op), ya que los registros no están expuestos como puertos.

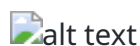
La secuencia recorre tres cargas encadenadas, r_a = 10 , r_b = 5 y r_op = 100000 (ADD), activando en cada paso un único enable. Después de la tercera carga la salida debe valer 15, lo que confirma que la ALU está correctamente conectada a los tres registros. El último paso es el complemento de los anteriores, se bajan las tres señales de carga y se cambian los switches a 99, para verificar que los registros retienen su valor y que ningún dato entra sin su enable correspondiente.

6.3 Resultados

En el tb_alu los 804 casos se ejecutaron sin diferencias contra el modelo de referencia:

alt text

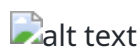
En el tb_alu_top la consola muestra las tres cargas independientes, la salida o_leds = 15 correspondiente a 10 + 5, y la retención de los tres registros con los enables en cero a pesar del cambio de los switches:

alt text

7. Síntesis e implementación

7.1 Impacto de la optimización del desplazador

Para cuantificar el efecto de acotar la cantidad de desplazamiento (sección 4.2), se sintetizó el módulo alu con y sin la máscara utilizando el sintetizador Yosys, que reporta el conteo en compuertas genéricas:

alt text

Versión	Celdas
Sin máscara (i_b completo)	316
Con máscara (i_b[NB_SHIFT-1:0])	263
Reducción	53 celdas (16,8 %)

La diferencia corresponde a las cinco etapas de cada desplazador de barril que la máscara elimina.

Además la reducción de etapas acorta el camino combinacional, lo que se traduce en un menor retardo de propagación.

7.2 Análisis de tiempo

El reloj se restringió desde el archivo de constraints con el período del oscilador de la placa:

```
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports i_clk]
```

Sobre el diseño en Vivado se corrió el `Report Timing Summary`, que no reporta ninguna violación:

 alt text

El slack (tiempo disponible - tiempo consumido) de setup y de hold aparece como infinito porque el diseño no tiene ningún camino registro a registro. Los tres registros alimentan una ALU puramente combinacional cuya salida va directo a los pines, así que todos los caminos empiezan o terminan en un puerto y quedan sin restringir al no haberse declarado `set_input_delay` / `set_output_delay`. Lo único que se verifica contra el reloj es el ancho de pulso, con 4,5 ns de margen sobre los 5 ns del semiperíodo.

Por eso el dato relevante no es el slack sino el retardo del camino crítico, que va desde el registro `r_b` hasta el LED más significativo atravesando la cadena de acarreo del sumador. Para aislarlo se pidió el peor camino de todos los que terminan en los pines de salida:

```
report_timing -to [get_ports o_leds*] -delay_type max -max_paths 1
```

 alt text

La columna `Path(ns)` del reporte acumula desde el flanco del reloj, por eso termina en 16,962 ns. Los primeros 5,158 ns son la distribución del clock desde el pin W5 hasta el registro (IBUF, BUFG y ruteo), común a todos los caminos. El retardo del dato propiamente dicho son los 11,803 ns restantes, que se reparten así:

Tramo del camino crítico	Retardo
Clock-to-Q del registro (FDRE)	0,518 ns
Lógica de la ALU y ruteo interno (LUT2, 2×CARRY4, LUT6, MUXF7)	7,590 ns
Buffer de salida hacia el pin (OBUF)	3,695 ns
Total	11,803 ns

De esos 11,803 ns, 3,695 ns corresponden al buffer de salida hacia el pin físico, que no forma parte de ningún lazo sincrónico y no condiciona la frecuencia de trabajo, menos aún cuando el destino son LEDs. El tramo que sí importa es el interno, 8,108 ns desde el flanco de clock hasta la entrada del buffer, por debajo de los 10 ns del período. Es decir que si el resultado de la ALU se registrara en lugar de salir directo a los pines, el diseño seguiría cerrando timing a 100 MHz con alrededor de 1,9 ns de margen.

8. Conclusiones

Se implementó y verificó una ALU combinacional de ocho operaciones, parametrizable en el ancho del bus de datos, que queda disponible como bloque reutilizable para los trabajos siguientes de la materia. La validación se hizo en dos etapas, primero por simulación, con un test bench de estímulos aleatorios que compara la salida contra un modelo de referencia sobre 804 casos, y después sobre la Basys3, donde el comportamiento observado en los LEDs coincidió con el de la simulación.

Más allá del resultado funcional, el trabajo dejó algunas conclusiones sobre las decisiones de diseño:

- La declaración `signed` de los puertos es un detalle importante ya que es lo que separa SRA de SRL. Sin ella, ambas operaciones son iguales.
- Acotar la cantidad de desplazamiento a `$clog2(NB_DATA)` bits es una decisión de especificación que permite darle uso al desplazamiento. Como beneficio adicional, redujo un 16,8 % la cantidad de celdas del diseño.
- El valor `x` en la rama `default` cumple dos funciones opuestas según el contexto en el que se encuentre, en simulación muestra de inmediato un opcode no contemplado, y en síntesis le da libertad al optimizador para elegir la implementación más económica.
- Parametrizar desde el principio tuvo un costo bajo y es lo que permite llevar el módulo a 16 o 32 bits sin modificar la lógica, que es la condición para poder reutilizarlo más adelante.